Aerial Robotics Kharagpur Documentation
Task 2.1 – Drone Line Follower

Parv Chopra *

***Abstract*—This report presents an image-based line-following algorithm for a Parrot Mambo minidrone. The drone's downward-facing camera provides 120×160 RGB frames; the algorithm detects a red line and outputs a unit steering vector (X, Y) to guide the drone along the line. Three approaches were developed: a 7×7 grid-based scanner, a 3-strip tangent method, and a global+perpendicular centroid method with heading-aware blending. The final algorithm handles turns exceeding $100°$ with no directional bias, operates entirely with fixed-size arrays for Simulink compatibility, and maintains a persistent heading to prevent oscillation.**

I. Introduction

The objective of Task 2.1 is to steer a drone so that it follows a red line painted on the ground. The input to the algorithm is three 120×160 `uint8` arrays (R, G, B channels from the downward camera), and the output is a unit vector (X, Y) where X points upward in the image (forward in the drone's frame) and Y points rightward.

Key challenges include:

- **Sharp turns** exceeding $100°$ where the line bends sharply.
- **Simulink compatibility**: the algorithm runs inside a MATLAB Function block, so all arrays must be compile-time fixed-size — no `find()`, `sort()`, `diff()`, or variable-length logical indexing.
- **Direction independence**: the algorithm must treat all headings equally with no built-in "north = forward" assumption.
- **Oscillation prevention**: a purely per-frame algorithm (no memory) would jitter on straight lines or flip at corners, so some form of frame-to-frame state is needed.

II. Problem Statement

Given per-frame R, G, B arrays of size 120×160 (`uint8`), detect a red line in the image and output a unit vector (X, Y) indicating the direction the drone should steer to follow the line. The algorithm must:

1) Detect red pixels reliably using thresholds: $R > 180$, $G < 100$, $B < 100$.
2) Determine the line's direction relative to the drone.
3) Handle turns from gentle curves to sharp corners exceeding $100°$.
4) Operate with only fixed-size arrays (Simulink constraint).
5) Be direction-independent — no axis is privileged as "forward."

III. Related Work

Centroid-based line following is common in ground robots — compute the centroid of detected line pixels and steer toward it. Simple but fails at sharp turns where the centroid averages both branches.

Hough Line Transform detects line parameters (ρ, θ) but requires variable-size arrays and is computationally heavy for real-time drone control.

Strip-based methods divide the image into horizontal strips and compute per-strip centroids. The tangent between adjacent centroids gives the line direction. Used in many line-following competitions.

PID controllers are often used downstream of the vision algorithm to smooth steering. Not used here since the output is a direction vector, not a motor command.

IV. Initial Attempts

A. Attempt 1: 7×7 Grid Scanner (`lineFollower.m`)

The image was cropped to 119×119 and divided into a 7×7 grid of 17×17 blocks. Each block was classified as "line present" if the red pixel count exceeded a threshold. Starting from the centre block $(4, 4)$, a compass-based scan searched for the next line block in 8 directions, prioritising the last known heading.

Algorithm:

1) Centre-crop the 120×160 image to 119×119 (rows 1–119, columns 21–139). Since $119 = 7 \times 17$, each grid block is exactly 17×17 pixels.
2) Create a binary red mask: $R > 180 \wedge G < 100 \wedge B < 100$.
3) Count red pixels in each of the 49 blocks.
4) Define 8 compass directions (N, NE, E, SE, S, SW, W, NW) mapping to the ring-1 neighbours of centre block $(4, 4)$.
5) Scan ring-1 blocks in priority order based on `lastDir`, checking forward directions first:

$$s(p) = ((d - 1 + b(p)) \bmod 8) + 1 \tag{1}$$

   where s = `scanOrder`, d = `lastDir`, b = `baseOff`, where `baseOff` $= [0, 1, 7, 2, 6]$ restricts scanning to the forward $\pm 90°$ cone.
6) The first block exceeding threshold $T = 0.10 \times 17 \times 17 \approx 29$ pixels wins.
7) Steer toward the chosen block's geometric centre.

Strengths: Simple, intuitive, works for moderate turns.

Weaknesses:

- Quantised to 8 directions (45° resolution) — poor angle accuracy.
- Initial forward-cone bug included $\pm 135^\circ$ directions, causing U-turns (fixed by limiting to $\pm 90^\circ$).

B. Attempt 2: 3-Strip Tangent Method (`lineFollower2.m`)

The image was divided into 3 horizontal strips of 40 rows each. The column centroid of red pixels in each strip was computed, and the tangent vector between strip centroids gave the line direction.

Strip layout:

- Strip A (far): rows 1–40, centroid row $= 20$.
- Strip B (mid): rows 41–80, centroid row $= 60$.
- Strip C (near): rows 81–120, centroid row $= 100$.

Steering logic:

- If both far and near strips are visible: $\vec{s} = (80,\ c_{\text{far}} - c_{\text{near}})$.
- If only mid and near: $\vec{s} = (40,\ c_{\text{mid}} - c_{\text{near}})$.
- If only near: lateral correction $\vec{s} = (40,\ c_{\text{near}} - 80)$.

The output angle is:

$$\theta = \arctan\left(\frac{c_{\text{far}} - c_{\text{near}}}{80}\right) \tag{2}$$

giving exact (unquantised) angular accuracy from the column-centroid difference over the 80-row strip separation.

Strengths: Much better angle accuracy ($\sim 5^\circ$) than the grid method. Simple and fast.

Weaknesses: Struggles on steep turns ($> 60^\circ$) where the line runs nearly vertically — horizontal strips see diminishing lateral offset, and fails entirely beyond $\sim 90^\circ$.

C. General Observation

An unexpected pattern across both attempts: restricting the field of view to a small region immediately ahead of the drone made steering easier, much like a ground robot with only a few colour sensors in a line in front of it. With a small, local window the mapping from sensory input to steering output is almost trivial — the line is either left, centre, or right. With the full 120×160 image, the algorithm sees both the old and new branches at corners, the line's curvature, and noise, all at once. Naively using more data made the problem harder rather than easier, because averaging or scanning over the full image mixes conflicting directional cues. Rather than falling back to a local-only approach, I chose to treat this as a challenge and attempt to make the global data work. The final approach (Section V) projects all pixels relative to the current heading to separate useful signal from noise, but due to time constraints the algorithm was not refined enough to fully match the robustness of simpler local methods at extreme turn angles.

V. Final Approach

A. Algorithm: Global + Perpendicular Centroid with Heading-Aware Blending (`lineFollower3.m`)

The final algorithm computes two heading-relative centroids and blends them based on how much of the line is ahead vs. behind. A global centroid serves as a fallback when data is insufficient.

Step 1 — Red Mask. A binary 120×160 mask of red pixels:

$$m_{ij} = \mathbb{1}[R_{ij} > 180 \wedge G_{ij} < 100 \wedge B_{ij} < 100] \tag{3}$$

Step 2 — Global Centroid. Compute the centroid of all red pixels using column/row projection sums:

$$c_{\text{col}} = \frac{\sum_j \left(\sum_i m_{ij}\right) \cdot j}{N} \tag{4}$$

$$c_{\text{row}} = \frac{\sum_i \left(\sum_j m_{ij}\right) \cdot i}{N} \tag{5}$$

where N is the total number of red pixels.

Step 3 — Ahead Centroid. For each pixel, compute its signed projection onto the persistent heading vector $(\texttt{dirX}, \texttt{dirY})$:

$$\texttt{proj}(i,j) = (c_r - i)\,\texttt{dirX} + (j - c_c)\,\texttt{dirY} \tag{6}$$

where $(c_r, c_c) = (60, 80)$ is the image centre. Pixels with $\texttt{proj} > 0$ are "ahead." The weighted centroid of $\texttt{mask} \cdot \max(0, \texttt{proj}+1)$ gives the ahead centroid. On straight lines, this dominates steering.

Step 4 — Perpendicular Centroid. Compute each pixel's perpendicular distance from the heading axis:

$$d_\perp(i,j) = (c_r - i)(-\texttt{dirY}) + (j - c_c)\,\texttt{dirX} \tag{7}$$

Weight by $|d_\perp|$ to emphasise pixels far from the heading line. At a sharp corner ($> 90^\circ$), the new branch sticks out sideways giving high perpendicular weight, so this centroid points toward the new branch.

Step 5 — Adaptive Blending.

$$\beta = 1 - \frac{\texttt{aheadTot}}{\texttt{totalRed}} \tag{8}$$

$$\vec{t} = (1-\beta)\,\vec{p}_{\text{ahead}} + \beta\,\vec{p}_\perp \tag{9}$$

On straight lines, $\beta \approx 0$ so the ahead centroid dominates. At sharp corners, $\beta \approx 0.5$–0.8 so the perpendicular centroid (new branch) takes over.

Step 6 — Steering Vector. From the image centre to the blended target:

$$\vec{s} = \begin{pmatrix} c_r - t_{\text{row}} \\ t_{\text{col}} - c_c \end{pmatrix}, \quad \hat{s} = \frac{\vec{s}}{|\vec{s}|} \tag{10}$$

Step 7 — Smooth Heading Update. The persistent heading blends 70% new output with 30% old heading:

$$\vec{d}_{\text{new}} = \frac{0.7\,\hat{s} + 0.3\,\vec{d}_{\text{old}}}{|0.7\,\hat{s} + 0.3\,\vec{d}_{\text{old}}|} \tag{11}$$

This provides smooth transitions and prevents abrupt heading reversals.

B. Hyperparameters

TABLE I
ALGORITHM HYPERPARAMETERS.

Parameter	Value	Description
Red threshold	$R>180, G<100, B<100$	Red pixel detection
α (blend)	0.7	New output weight
Min red pixels	15	Keep last heading below

VI. RESULTS AND OBSERVATION

Table II compares the three approaches across different turn severities.

TABLE II
COMPARISON OF THE THREE LINE-FOLLOWING METHODS.

Method	Straight	90°	100°+	Angle Acc.
7×7 Grid	✓	✓	×	$\sim 45°$
3-Strip Tangent	✓	✓	×	$\sim 5°$
Global+Perp Blend	✓	✓	✓	$\sim 5°$

The 7×7 grid method suffered from $45°$ direction quantisation and could not handle sharp turns beyond $90°$. The 3-strip tangent method gave better angular accuracy by computing exact column-centroid differences, but its reliance on horizontal strips caused degraded performance on steep turns, failing entirely beyond $\sim 90°$.

The final method avoids both problems by computing heading-relative projections over the full image rather than fixed strips or a discrete grid. The adaptive blending between ahead and perpendicular centroids lets the algorithm follow straight lines normally (ahead centroid dominates) while switching to the perpendicular centroid at sharp corners where the new branch sticks out sideways. The persistent heading with $70/30$ blending prevents oscillation on straight sections while still permitting rapid direction changes at corners.

Key observations:

- The perpendicular centroid is critical for $> 90°$ turns — without it, the ahead centroid points backward along the old branch.
- The β blending ratio naturally increases as the turn sharpens, providing a smooth transition without explicit corner detection.
- The minimum red pixel threshold (15) was not needed for this project's controlled environment, but was included as a safeguard for real-world scenarios where the line may be partially occluded or poorly lit.

The $\sim 100°$ turn limit is not ideal — real tracks can have sharper corners and even U-turns. This represents the best result achieved within the available development time; the approach of using global image data made the problem harder than expected (see Section IV-C), and further refinement of the blending logic or a fundamentally different corner-handling strategy would be needed to push beyond this limit.

VII. FUTURE WORK

- **Adaptive thresholding**: The fixed $R > 180/G < 100/B < 100$ threshold may fail under different lighting conditions. An adaptive threshold based on histogram analysis could improve robustness.
- **Predictive steering**: Using velocity estimates to predict where the line will be 2–3 frames ahead, enabling smoother anticipation of turns.
- **T-junction handling**: The current algorithm does not handle T-intersections. Adding fork detection and branch commitment logic would be needed for more complex track layouts.
- **PID-based smoothing**: Integrating a PID controller on top of the vision output could reduce residual jitter.
- **Angular clamping**: Adding a circular history buffer of past output directions and rejecting outputs that deviate $> 90°$ from any recent direction could prevent U-turns at extreme corners.
- **Sharp and U-turn handling**: The current algorithm fails beyond $\sim 100°$. Handling turns up to $180°$ (full U-turns) is an important next step — this likely requires detecting that the line doubles back and committing to the new branch rather than blending with the old one.

CONCLUSION

A direction-independent, fixed-size-array-compatible line-following algorithm was developed for the Parrot Mambo minidrone. The final approach uses ahead and perpendicular centroid blending with a persistent heading to handle turns exceeding $100°$. Three approaches were explored — a 7×7 grid scanner, a 3-strip tangent method, and the final centroid-blending method — each iteration fixing failure modes found in the previous one. The algorithm is fully compatible with Simulink MATLAB Function blocks and requires no variable-size operations.